

FROGBYTE HPC F25

ANNA NORRIS
CARMEN PARK
DAVID HADDAD
STELLA GREENVOSS

OCTOBER 27TH, 2025

WITH JEE CHOI

1 Proposal

Sparse Matrix-Vector (SpMV) multiplication is a fundamental operation in scientific computing and graph analytics. It efficiently computes the product $y = Ax$ of a sparse matrix A and a vector x by performing operations only on the nonzero elements. This significantly reduces the computational cost compared to dense matrix operations. Optimizing SpMV is also multifaceted in that there are numerous SPMV computational kernels and routines within larger processes that each have their own parallelization requirements.

Our project will examine current SpMV formats (ELL, COO, CSR, and possibly HYB) and their respective kernels. Which kernel-pair format performs best for different matrix types? Similarly, at what point does a combination or hybrid approach outperform its more pure counterparts¹? Our implementation will analyze existing

algorithms by comparing each kernel against a diverse set of matrix sparsity patterns, including regular, irregular, and extremely irregular patterns, as well as uniform versus skewed density distributions. Our benchmarks and comparisons will be run on the University of Oregon's High Performance Computing cluster Talapas, with a particular focus on the scalability of each format.

1.1 Research on Common Formats

SpMV is a core operation in computer science, with applications in engineering, physics, and other fields. This broad applicability, coupled with the noticeable memory bandwidth bottleneck present when performing operations on large matrices, necessitates a suite of highly optimized, tailored kernels for computing SpMV.

As noted before, matrices can have a variety of sparsity patterns. Regu-

¹This will depend on whether we have time to work on this task.

lar patterns refer to a matrix where a sub-pattern can be found in the distribution of nonzero numbers within the matrix. Conversely, irregular patterns and highly irregular patterns refer to a lack of such a pattern. In some sparse matrices, more non-zero numbers may be found within a specific quadrant or axis than others; these will be referred to as skewed density distributions, while uniform density describes an even distribution of nonzero numbers within the matrix.

We propose implementing, benchmarking, and analyzing these three distinct SpMV kernels given a variety of sparse matrix patterns.

The code for this project will be based in C++ to take full advantage of the low level hardware control, like memory utilization, as well as its object oriented abstraction that lends itself to linear algebra constructs. It will be extended with NVIDIA’s CUDA parallel computing platform as CUDA is the industry standard for GPU computing. To ensure a rigorous and meaningful evaluation, we will be using NVIDIA’s cuSPARSE library as the baseline for performance. It is already quite optimized and it will help in to precisely identifying and overcoming specific performance limitations.

The matrices that will be selected for our dataset will span multiple areas of computer science and math, including finite-difference methods, eigenvalue computation, web graphs, and social networks. This is an effort to ensure our kernels are tested on regular, irregular, and skewed matrices with diverse properties to prevent niche op-

timization for only specific cases. For instance, we plan on using web graphs from SNAP, which provides network datasets from large corporations like Google and Twitter/X. Each dataset will be obtained as an .mtx file from the SuiteSparse Matrix Collection at <https://sparse.tamu.edu/> [5], which is stored in COO by default. We will then read and parse each .mtx file into each storage format using our program. All datasets we use will be publicly available to permit study replication.

2 Custom Kernels

For all of the kernels, we will use a CPU (AMD Milan 128 Cores, 512 GB RAM) node and GPU (52 A100 GPUs, 48 Cores, 256/512 GB RAM) nodes on the Talapas system and scale as much as we can, increasing the number of nodes used as resources/scheduling permits.

COO The coordinate format (COO) maps a sparse matrix by the row and column positions of nonzero numbers using two arrays, with an additional array holding the data entries. Position i in the row, column, and data arrays refers to one non-zero number in the sparse matrix. In this case, atomic operations, memory access patterns, and load imbalances are liable to cause poor thread utilization and thus are major bottlenecks for COO. Flegar [3] utilizes a COO kernel system that efficiently generates SpMV outputs for highly irregular matrices.

- **Kernel 1A: COO “Scalar” (Naive)**

The most straightforward implementation of COO where each

thread processes one nonzero element. This method leads to every element requiring an atomic operation and massive congestion when multiple threads hit the same row. There is little to no parallelism in this instance, as almost all of the operations inadvertently become serialized regardless of multithreading.

- **Kernel 1B: COO “Warp” (Optimized)**

Key optimizations from Fleegar [3] utilize chunking, register accumulation, and double sorting, where the first sort reduces atomic operations from $O(\text{number of nonzeros})$ to $O(\text{rows})$, and the secondary sort transforms random vector access to sequential, overall optimizing memory utilization. Each warp, meaning a group of 32 threads on a GPU, processes one chunk together. Optimizations address naive COO issues and potentially prove useful for irregular matrices, whereas CSR or ELL would lead to load imbalances.

CSR Compressed Sparse Row (CSR) compresses row information by only storing the start pointers for each row, the column indices, and data values in three separate arrays. In the case of storage, COO requires storage space equal to three times the number of nonzero entries, while CSR only uses around two times the number of nonzero entries, making it much more memory-efficient [2]. It also has more opportunities for load balancing and vectorization, making it a preferred choice for regular sparse patterns.

- **Kernel 2A: CSR “Scalar” (Naive)**

This kernel will assign one CUDA thread for each matrix row. Research has shown that this tends to perform poorly due to workload imbalance in irregular matrices. This implementation will serve as our starting point for familiarizing ourselves with CUDA commands and GPU programming before more complex, potentially hybrid kernels further down the road. It may also be good as a baseline to compare performance with other kernels.

- **Kernel 2B: CSR “Warp” (Optimized)**

A CUDA warp is assigned to each row. The increased number of threads and the way the warp connects them enables collaboration on the partial products. Since it increases thread-level parallelism, this approach is typical for high-performance versions of CSR.

ELL ELLPACK (ELL) regularizes matrices into fixed-width rows that hold value and column information. The width of the storage matrices are equal to the greatest number of nonzero values in the original matrix. For example, the ELL storage format breaks a matrix M into two matrices, which we can call D and I . D stores the nonzero data, while I stores the column indices for each datum. As described above, ELL’s efficiency fails when the row with the greatest number i of nonzero entries is an outlier such that i is much greater than the average number (j) of nonzero entries

in the rows of M . This hybrid format can instead store the j nonzero values for each row using ELL, placing the outlying data inside the COO structure. This should lead to better storage and access efficiency. Thus, this format is most efficient for storing matrices whose maximum number of nonzeros in a row is close to the average; matrices with significant differences between these two values should be stored in a different format.

- **Kernel 3: ELLPACK (ELL)**

A kernel where a single CUDA thread is assigned to each row. Since the data is stored in a two-dimensional array, all threads in a warp can execute in lockstep and perform coalesced memory accesses.

HYB Hybrid (HYB) can be a combination of any two SPMV formats. For the purposes of this project, HYB will refer to a combination of the ELL/COO formats. Bell and Garland describe a hybrid format that stores typical nonzeros in the ELL format and exceptional rows in the COO format. They claim that this maintains the relative performance gains offered by ELL while protecting it from performance drops caused by outliers to which ELL is susceptible. Li et al. [6] proposes an alternative hybrid kernel utilizing CSR and ELL formats. Their method partitions the matrix based on row characteristics. Long irregular rows are stored using CSR to avoid excessive zero padding that ELL requires in such cases. Shorter, uniform rows are stored in ELL for maximum case specific efficiency that it was created for [6]. These hybrid models allow for broader scalability to commercial size

graphs and other matrices.

- **Kernel 4: Hybrid HYB (Time permitting)**

This would be a high-level approach that splits matrices into a section evaluated using an ELL kernel and a section evaluated with a COO kernel, dependent on factors including the distribution of sparsity in the matrix and how irregular the rows are.

2.1 Benchmarking and Analysis

For each matrix type (regular, irregular, highly irregular), we will run all custom kernels (CSR, COO, ELL, and time permitting HYB) and the corresponding cuSPARSE SpMV kernel as our baseline. This will measure the following:

- **Correctness:** All results will be verified by comparing the output vector from our custom GPU kernels against a serially-computed result from CPU-based implementation. This is to ensure numerical accuracy of results before analyzing performance.
- **Preprocessing and Conversion (Overhead):** We will measure the wall-clock time required for initial data parsing (from the '.mtx' file) and the conversion from the default COO format to each target storage format (CSR, ELL). This is an often overlooked, but essential part of the total runtime for implementations/formats that require this overhead computation.

- **Kernel Performance (GFLOPS and GB/s):** We will isolate the kernel-only execution time to calculate:

- **Performance in GFLOPS):** Calculated as

$$\frac{(2 \times nnz)}{kerneltime}$$

- **Effective Memory Bandwidth in GB/s:** Calculated as

$$((bytesforA) + (bytesforx) + (bytesfory))/kerneltime$$

Since SpMV implementations are almost always bound by memory-bandwidth, this will be the primary metric for determining success of optimizations.

- **Scalability Analysis:** Using the Talapas cluster will allow for us to perform a scaling analysis. Execution benchmarks will be performed on a single problem size with an increasing number of GPU nodes. This will allow for measurement of speedup and efficiency to evaluate how each kernel utilizes the distributed system’s cluster resources.

less impact on execution time.

The warp versions of CSR and COO will perform better than their naive counterparts. We expect ELL to perform better on uniform matrices based on the research we have done. If we are able to complete and test the hybrid kernel, we expect to find that HYB has the best overall performance.

We expect to see better performance as the number of cores utilized increases; it will be interesting to compare the points at which performance stops scaling for each kernel.

2.2 Expected Results

CSR might scale better on Talapas because it is more memory-efficient than COO, meaning that memory-related overhead from the cluster may accrue

References

- [1] Bell, N. & Garland, M. (2008). Efficient Sparse Matrix-Vector Multiplication on CUDA. *Nvidia Technical Report NVR-2008-004* Vol. 2. No. 5.
- [2] Benatia, A., Ji, W., Wang, Y., & Shi, F. (2018). BestSF: A Sparse Meta-Format for Optimizing SpMV on GPU. *ACM Transactions on Architecture and Code Optimization*, 15(3), 29. <https://doi.org/10.1145/3226228>
- [3] Flegar, G., & Anzt, H. (2017). Overcoming Load Imbalance for Irregular Sparse Matrices. In *Proceedings of the International Conference on Supercomputing (ICS)*, Chicago, IL, USA. <https://publikationen.bibliothek.kit.edu/1000079215>
- [4] Gao, J., Wang, Y., & Wang, J. (2016). A novel multi-graphics processing unit parallel optimization framework for the sparse matrix-vector multiplication. *Concurrency and Computation: Practice and Experience*, 29(17), e3936. <https://doi.org/10.1002/cpe.3936>
- [5] Kolodziej, S. P., Aznaveh, M., Bullock, M., David, J., Davis, T. A., Henderson, M., Hu, Y. & Sandstrom, R. (2019). The SuiteSparse Matrix Collection Website Interface. *Journal of Open Source Software* Vol. 4. No. 35. pages 1244.
- [6] Li, K., Yang, W., & Li, K. (2015). Performance Analysis and Optimization for SpMV on GPU Using Probabilistic Modeling. *IEEE Transactions on Parallel and Distributed Systems*, 26(1), 196-205. <https://doi.org/10.1109/TPDS.2014.2308221>
- [7] Tsai, Y. M., Cojean, T., & Anzt, H. (2021). Sparse Linear Algebra on AMD and NVIDIA GPUs – The Race Is On. In *Proceedings of the International Conference on High Performance Computing (ISC)*, Virtual Event. https://doi.org/10.1007/978-3-030-78713-4_11